Laboratory 4
Semantic segmentation

Objectives

In this laboratory, we study semantic segmentation, the task of assigning a label to every pixel in an image. Unlike image classification, which compresses an image into a single prediction, segmentation needs to keep track with object extent, boundaries, and context. The objectives of this lab are to understand how segmentation models are built, why upsampling is required, how segmentation performance is measured, and how different losses and architectural choices influence the quality of the predicted masks.

Proposed problems

---

1. Datasets and Dataloaders

In the first part of the laboratory, we'll focus on data and data loading. We'll be working with the Labeled Faces in the Wild dataset which contains facial images, and each pixel in the image is annotated with the following three classes: background, skin, and hair. The dataset with the segmentation masks can be found here. Alternatively, if you have more computing power, you can work with CelebA-MaskHQ dataset.

Up to now, we could get away with loading a dataset all at once in memory: read every image into a big array/tensor, shuffle it, and feed it to the model. That approach is fine for small toy datasets, but it breaks quickly in real settings, where images are large, datasets are huge). The standard workflow is therefore lazy loading: keep the data on disk, and load only the samples you need.

In PyTorch, this workflow is split into two concepts:

- A ***Dataset*** answers the question: "Given an index i, how do I get the *i*-th training sample?". It defines the mapping from an integer index to actual data on disk (image file + segmentation mask file), plus any preprocessing/augmentation you want to apply.
- A ***DataLoader*** answers the question: "How do I efficiently iterate over the dataset during training?" It handles batching (grouping samples into batches), shuffling, parallel loading using worker processes, and collating individual samples into a batch tensor.

The Dataset describes what a sample is, and the DataLoader describes how we iterate over samples.

Each custom dataset must implement the following methods:

- __init__ : the constructor is run when instantiating the Dataset object. Here you should do the initializations (input dirs, dataset splits) and the transforms (covered in more detail in the next section).
- __len__ : this should return the number of samples in the dataset;

- __getitem__ : this should load and return a sample from the dataset at the given index idx (passed as a parameter). Based on the index, it identifies the image and its corresponding segmentation map location on disk, calls the transform functions on them (if applicable), and returns the tensor image and mask in a tuple.

Below you have the main structure of the ***LFWDataset*** dataset class that allows you to download programatically the images (you should consider moving the download helper functions in another module).
!!!Attention not all input images have corresponding segmentation masks!!!

```
import os
import torch
import hashlib
import tarfile
import requests
from tqdm import tqdm


class LFWDataset(torch.utils.data.Dataset):
    _DATA = (
        # images
        ("http://vis-www.cs.umass.edu/lfw/lfw-funneled.tgz", None),
        # segmentation masks as ppm
        ("https://vis-www.cs.umass.edu/lfw/part_labels/parts_lfw_funneled_gt_images.tgz",
         "3e7e26e801c3081d651c8c2ef3c45cfc"),
    )


    def __init__(self, base_folder, transforms, download=True, split_name: str = 'train'):
        super().__init__()
        self.base_folder = base_folder
        # TODO your code here: if necessary download and extract the data
        # TODO based on the split_name get the paths of the samples and their corresponding ground truth
        if download:
            self.download_resources(base_folder)
        self.X = None
        self.Y = None
        raise NotImplementedError("Not implemented yet")

    def __getitem__(self, idx):
        # TODO your code here: return the idx^th sample in the dataset (image, segmentation mask)
        # TODO your code here: if necessary apply the transforms
        raise NotImplementedError("Not implemented yet")

    def download_resources(self, base_folder):
        if not os.path.exists(base_folder):
            os.makedirs(base_folder)
        self._download_and_extract_archive(url=LFWDataset._DATA[1][0], base_folder=base_folder,
                                           md5=LFWDataset._DATA[1][1])
```

```
        self._download_and_extract_archive(url=LFWDataset._DATA[0][0], base_folder=base_folder,
md5=None)

    def _download_and_extract_archive(self, url, base_folder, md5) -> None:
        """
          Downloads an archive file from a given URL, saves it to the specified base folder,
          and then extracts its contents to the base folder.

          Args:
          - url (str): The URL from which the archive file needs to be downloaded.
          - base_folder (str): The path where the downloaded archive file will be saved and extracted.
          - md5 (str): The MD5 checksum of the expected archive file for validation.
          """
        base_folder = os.path.expanduser(base_folder)
        filename = os.path.basename(url)

        self._download_url(url, base_folder, md5)
        archive = os.path.join(base_folder, filename)
        print(f"Extracting {archive} to {base_folder}")
        self._extract_tar_archive(archive, base_folder, True)

    def _retreive(self, url, save_location, chunk_size: int = 1024 * 32) -> None:
        """
            Downloads a file from a given URL and saves it to the specified location.

            Args:
            - url (str): The URL from which the file needs to be downloaded.
            - save_location (str): The path where the downloaded file will be saved.
            - chunk_size (int, optional): The size of each chunk of data to be downloaded. Defaults to
32 KB.
            """
        try:
            response = requests.get(url, stream=True)
            total_size = int(response.headers.get('content-length', 0))

            with open(save_location, 'wb') as file, tqdm(
                    desc=os.path.basename(save_location),
                    total=total_size,
                    unit='B',
                    unit_scale=True,
                    unit_divisor=1024,
            ) as bar:
                for data in response.iter_content(chunk_size=chunk_size):
                    file.write(data)
                    bar.update(len(data))

            print(f"Download successful. File saved to: {save_location}")

        except Exception as e:
            print(f"An error occurred: {str(e)}")
```

```python
    def _download_url(self, url: str, base_folder: str, md5: str = None) -> None:
        """Downloads the file from the url to the specified folder

        Args:
            url (str): URL to download file from
            base_folder (str): Directory to place downloaded file in
            md5 (str, optional): MD5 checksum of the download. If None, do not check
        """
        base_folder = os.path.expanduser(base_folder)
        filename = os.path.basename(url)
        file_path = os.path.join(base_folder, filename)

        os.makedirs(base_folder, exist_ok=True)

        # check if the file already exists
        if self._check_file(file_path, md5):
            print(f"File {file_path} already exists. Using that version")
            return

        print(f"Downloading {url} to file_path")
        self._retreive(url, file_path)

        # check integrity of downloaded file
        if not self._check_file(file_path, md5):
            raise RuntimeError("File not found or corrupted.")

    def _extract_tar_archive(self, from_path: str, to_path: str = None, remove_finished: bool = False)
-> str:
        """Extract a tar archive.

        Args:
            from_path (str): Path to the file to be extracted.
            to_path (str): Path to the directory the file will be extracted to. If omitted, the
directory of the file is
                used.
            remove_finished (bool): If True , remove the file after the extraction.
        Returns:
            (str): Path to the directory the file was extracted to.
        """
        if to_path is None:
            to_path = os.path.dirname(from_path)

        with tarfile.open(from_path, "r") as tar:
            tar.extractall(to_path)

        if remove_finished:
            os.remove(from_path)

        return to_path
```

```python
    def _compute_md5(self, filepath: str, chunk_size: int = 1024 * 1024) -> str:
        with open(filepath, "rb") as f:
            md5 = hashlib.md5()
            while chunk := f.read(chunk_size):
                md5.update(chunk)
        return md5.hexdigest()

    def _check_file(self, filepath: str, md5: str) -> bool:
        if not os.path.isfile(filepath):
            return False
        if md5 is None:
            return True
        return self._compute_md5(filepath) == md5


if __name__ == '__main__':
    LFWDataset(download=True, base_folder='lfw_dataset', transforms=None)
```

After you wrote the Dataset, you can use a Dataloader to iterate through the samples. Tips on how to improve the performance of Dataloaders can be found here.

```python
# let's create a DataLoader to easily iterate over this dataset
bs = 4
dataloader = torch.utils.data.DataLoader(training_data, batch_size=bs, shuffle=True, num_workers=0)

for i_batch, sample_batched in enumerate(dataloader):
    imgs = sample_batched[0]
    segs = sample_batched[1]
    print(i_batch, imgs.size(), segs.size())

    rows, cols = bs, 2
    figure = plt.figure(figsize=(bs, 2))
    for i in range(0, bs):
        figure.add_subplot(rows, cols, 2*i+1)
        plt.title('image')
        plt.axis("off")
        plt.imshow(imgs[i].numpy())

        figure.add_subplot(rows, cols, 2*i+2)
        plt.title('seg')
        plt.axis("off")
        plt.imshow(segs[i].numpy(), cmap="gray")
    plt.show()
    # display the first 3 batches
    if i_batch == 2:
```

```
        break
```

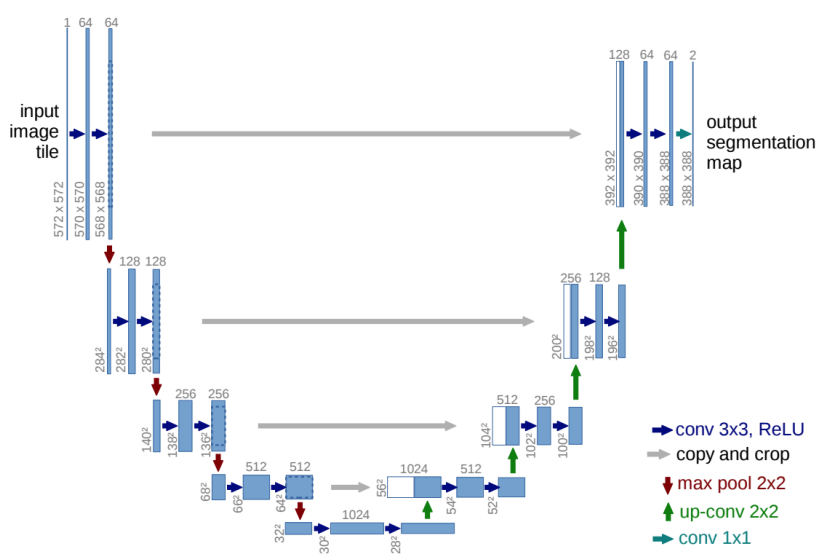
Each image is associated with a mask of the same spatial resolution, where every pixel stores a class index. This makes data augmentation more delicate than in classification, because any spatial operation must preserve alignment between image and mask.
So, if you are using an augmentation that implies some kind of geometric transformation (like crops, warp affine, rotate, etc.), the same transformation must be applied identically to images and masks. A random crop or flip changes pixel locations, and applying it to the image but not to the mask breaks supervision.
Implement from scratch an paired geometric augmentation. . Then, you will use torchvision.transforms.**v2**, which provides mechanisms for applying synchronized transformations to images and masks.

2. Image segmentation models

The model that will be used in this laboratory is inspired by the U-Net architecture. U-Net is a fully convolutional neural network comprising two symmetric paths: a **contracting** path (to capture context) and an **expanding** path (which enables precise localization). The network also uses skip connections between the corresponding layers in the downsampling path to the layer in the upsampling path, and directly fast-forwards high-resolution feature maps from the encoder to the decoder network.

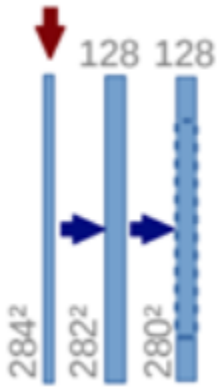
The output of the model is a volume with depth C, where C is the number of pixel classes. For example, if you want to classify the pixels into person and background, the output will be a volume of depth 2. If you want to classify the pixels into face, hair and background the output will be a volume of depth 3.

Read the U-Net paper and try to understand the architecture. Of course, you are encouraged to check out newer CNN architectures and try to reproduce them, but for pedagogical purposes we'll stick to UNET.
An overview of the U-Net architecture is depicted in the figure below:



The contracting path

The contracting path contains the well known layers: convolutions and max pooling. Here you can either use a custom convolutional neural network, that you will write and train it from scratch, (lower performance and higher chance of overfitting), or use a pretrained torchvision model.
If you choose to **write the encoder from scratch**, write a class Encoder which inherits from torch.nn.Module. The model will be composed of several blocks, each block comprising two convolutional layers (with filter size of $k=3$) with a ReLu non-linearity between them. The blocks of the encoder will be separated by max pooling layers with a size of 2 and a stride of 2.



In the constructor of the Encoder class you will pass as parameter a list containing the number of channels used for each encoder block. In the forward function you will return a list containing the outputs of all the encoder blocks (you'll be using them for the connection between the down-sampling path and the up-sampling path).

If you choose to use a **pre-trained module from torchvision** for this, first load the pre-trained weights on ImageNet, and "freeze" these weights during the training process (set required_grad=False for those tensors), or you can train them with a smaller learning rate. The problem is that to create the skip connections required by the U-Net architecture we need access to the feature maps of some intermediate layers in the network and these are not accessible by default. For this, you should inspect the implementation of the CNN that you chose, make the Encoder class inherit from the model you chose, and in the forward function, return the output of the layers you selected for the skip connections.

The expanding path

In semantic segmentation, the network reduces spatial resolution as it goes deeper (because of pooling and strided convolutions).

Upsampling is the step where we bring those low-resolution activation maps back to a higher resolution, so the model can produce a dense, per-pixel prediction (a mask) aligned with the original image grid. There are multiple ways to upsample, and it's useful to separate them into two broad categories:

- *Unlearned upsampling* (fixed interpolation): the operation has no trainable parameters. CExamples are nearest-neighbor upsampling and bilinear interpolation. These are simple, fast, and stable, but they can't "learn" task-specific refinement on their own.

- *Learned upsampling* (trainable): the upsampling includes parameters that are optimized during training. An example is the transposed convolution (also called "deconvolution"), implemented in PyTorch as *torch.nn.ConvTranspose2d*.

Let's write a function to implement an upsampling block, consisting of a transposed convolution, a batch normalization block and a ReLu activation.
Remember, the output size Wo of a transposed convolutional layer is:

$$W_o=(W_i-1)\cdot S-2P+F$$

, where W_i is the size of the input, S is the stride, P is the amount of padding and F is the filter size.

```
def upsample_block(x, filters, size, stride = 2):
  """
  x - the input of the upsample block
  filters - the number of filters to be applied
  size - the size of the filters
  """

  # TODO your code here
  # transposed convolution
  # BN
  # relu activation
  return x

in_layer = torch.rand((32, 32, 128, 128))

filter_sz = 4
num_filters = 16

for stride in [2, 4, 8]:
  x = upsample_block(in_layer, num_filters, filter_sz, stride)
  print('in shape: ', in_layer.shape, ' upsample with filter size ', filter_sz, '; stride ', stride, '
-> out shape ', x.shape)
```

You will use these upsampling blocks in the expanding path.
Write a class **Decoder** which inherits from *torch.nn.Module* to implement the up-samping path; the constructor of this class will get as parameter the depth of each decoder module, and in the forward function you will have two parameters: the input feature map and a list of activations from the encoder (for the skip connections).
In the forward function, for each block:

- apply the up-sampling operation (followed by batch normalization and ReLU);
- crop the corresponding activation map from the encoder (use CenterCrop) such that is has the same size as the decoder block;
- concatenate these two activation maps (on the channel dimension);
- apply an encoder block on the result;
- pass the result to the next decoder block.

Finally, write the UNet class which defines the semantic segmentation model. Here, you will connect the Encoder and Decoder classes that you previously wrote. The output of the segmentation module will be a volume with as many channels as the number of classes from the segmentation problem. Basically, each channel ***c*** from the output will be a map that stores the probability of each pixel to belong to the class ***c***.

Apply a *1x1* convolution with c channels on the decoder output to obtain the segmentation map.
Finally, to have matching shapes between the network output (segmentation map) and the ground truth data, resize the segmentation map using *torch.nn.functional.interpolate*.

3. Loss functions for semantic segmentation

For semantic segmentation, the loss function must compare per-pixel predictions with per-pixel ground-truth labels. Unlike image classification, we are not predicting a single class per image, but a class for every pixel, which strongly constrains the choice of loss.

The most common choice is cross-entropy loss. In segmentation, cross-entropy is applied independently at each pixel: the model outputs a class distribution for every pixel, and the loss penalizes the mismatch between the predicted distribution and the ground-truth class label. This makes cross-entropy a natural and well-understood baseline. It is stable, easy to optimize, and works well when classes are reasonably balanced.

However, segmentation datasets often suffer from class imbalance (e.g. a large background region and small foreground objects). In such cases, cross-entropy alone may bias the model toward predicting the dominant class. This is where Dice loss becomes useful.

Dice loss is derived from the Dice similarity coefficient, which directly measures the overlap between the predicted mask and the ground-truth mask. For a single class, the Dice coefficient is defined as:

$$Dice = \frac{2 * \sum_i (p_i \cdot g_i)}{\sum_i p_i + \sum_i g_i}$$

, where p_i is the predicted probability for pixel *i* and g_i is the ground-truth label (usually 0 or 1). Dice loss is then defined as:

$$L_{Dice} = 1 - Dice.$$

Dice loss focuses on overlap quality rather than per-pixel classification accuracy. This makes it suitable for small objects and imbalanced foreground-background scenarios.
In practice, cross-entropy and Dice loss are often combined as:

$$L = L_{CE} + \lambda L_{Dice}.$$

This combination benefits from the stable optimization behavior of cross-entropy while encouraging better mask overlap via the Dice loss. The weighting factor λ controls the relative importance of the two terms.

Optionally, you can experiment with other loss functions, for example to improve the boundary quality. Losses such as Boundary loss, Hausdorff loss, or focal variants designed to highlight the contour pixels can improve edge sharpness and object separation.

4. Logging and Evaluation metrics

Implement several segmentation metrics to evaluate the model you've just trained. As usual, try to implement these metrics without using any for loops.

In the remainder of this section we'll use the following notation:

- n_{ij} - the total number of pixels classified to class j but actually belonging to class i; $i, j \in 1, .., C$;
- $t_i = \sum_{j=1}^{C} n_{ij}$ - the total number of pixels belonging to class i (in the ground truth segmentation mask);
- - C - the total number of classes in the segmentation problem.

Mean pixel accuracy
Pixel accuracy is the simplest image segmentation metric; it is defined as the percentage of pixels that were correctly classified by the model.

$$p_a = \frac{1}{C} \frac{\sum_i^C n_{ii}}{\sum_i^C t_i}$$

This metric is not that relevant for class imbalanced problems (which occurs for most segmentation problems).

Intersection over Union (IoU)
The intersection over union metric is defined as the ratio between the area of intersection and the area of union (between the predicted segmentation mask and the ground truth segmentation mask of a single class). In case of a multi-class segmentation problem, we simply average the IoUs over all the classes. This metric is called Mean Intersection over Union (mIou).

$$mIoU = \frac{1}{C} \sum_{i=1}^{C} \frac{n_{ii}}{t_i - n_{ii} + \sum_{j=1}^{C} n_{ji}}$$

The ideal value for this metric is 1; usually values lower than 0.6 indicate a very bad performance.

Frequency Weighted Intersection over Union

The frequency weighted over union metric is similar to mean IoU, but the values are weighted with the adequate frequencies of the pixels.

$$fIou = (\sum_{i=1}^{C} t_i)^{-1} \sum_{i=1}^{C} t_i \cdot \frac{n_{ii}}{t_i - n_{ii} + \sum_{j=1}^{C} n_{ji}}$$

The values of this metric lie in the interval [0, 1], and the ideal value for this metric is 1.
Compute and report these metrics for your trained models.

When logging your experiments, do not limit yourself to numerical metrics. **Semantic segmentation is a highly visual task, and many important failure modes are only visible when you look at the predicted masks.**

In addition to logging metrics in *wandb*, also visualize model predictions throughout training. Log examples where the predicted mask is overlaid on top of the input image, together with the ground-truth mask. This allows you to determine boundary quality, small object detection, and systematic errors that are not well captured by scalar metrics. Check this tutorial to see how you can achieve this.

5. Training the model

By now, the structure should be familiar from the previous labs. You will need two ***DataLoaders***: one for training and one for validation (or test). Before entering the loop, define the loss function and choose an optimizer.

For each training epoch, the loop should follow this structure:

- Get a batch of images and segmentation masks from the training DataLoader.
- Zero out the optimizer's gradients (gradients accumulate by default in PyTorch).
- Perform the forward pass through the model.
- Compute the loss and the metrics from the model predictions and the ground-truth masks.
- Run the backward pass and let the optimizer perform one update step on the model parameters.

As always, during training, log the relevant metrics to *wandb*. At a minimum, you should log training loss, training metrics, and after completing the training phase for an epoch, evaluate the model on the validation/test dataloader. Compute the validation loss and validation accuracy using the same definitions as in training.
The goal is not just to train the model, but to monitor how training and validation metrics evolve together.

6. Post-processing

The output of a semantic segmentation model is not a final mask, but a per-pixel probability distribution over classes. The predicted segmentation mask is usually obtained by taking an *argmax* over these probabilities, but this step discards useful information about confidence and uncertainty.

Post-processing works on the predicted masks or probabilities after training, without modifying the model. Its purpose is to enforce simple assumptions that the model does not explicitly learn.

Implement a post-processing technique that use the predicted probabilities. For example, you can remove low-confidence predictions by applying a confidence threshold, setting pixels with low maximum probability to background or to an "unknown" label. You can also smooth predictions spatially by averaging probabilities in a local neighborhood before taking the *argmax*.

Then implement another simple postprocessing, in which you remove small isolated regions that appear in the predicted mask. These regions often correspond to low-confidence predictions and can be filtered using connected-component analysis (you can use *opencv* for this).
Check how these preprocessing impacts the results.

To sum up, your tasks for this lab:

- Implement a custom segmentation Dataset and DataLoader, handling paired images and masks correctly.
- Write at least **one paired image-mask data augmentation from scratch** and then use torchvision.transforms.v2 for synchronized image-mask transforms.
- Implement a CNN segmentation model (UNet), which includes an encoder, decoder, skip connections, and upsampling layers.
- Implement and report segmentation metrics (pixel accuracy, IoU, mean IoU, frequency-weighted IoU) without explicit loops.
- Log training and evaluation results to wandb, including visualizations of predicted masks overlaid on input images.
- Make several trainings in which you:
    - Experiment with different upsampling strategies and analyze their effect on segmentation quality.
    - Train the model using pixel-wise cross-entropy, Dice loss, and a combined loss, and optionally explore an edge-aware loss.
- Apply post-processing to the predicted probability maps and masks (confidence thresholding, region filtering) and evaluate its impact.
- Make a report with all these results